1. A clear step-by-step description of your method, including pseudocode/algorithms for: (i)force/energy evaluation, (ii) time stepping, (iii) enforcement of Dirichlet constraints, and (iv) the path-planning or control law mapping the tracking error of node j to {xc, yc, θc}.:

(i) Force/Energy Evaluation

`objfun()` uses a discrete elastic-rod model: per edge it computes stretch; per interior vertex it computes curvature from adjacent unit tangents (`te`, `tf`). Axial (EA) and bending (EI) forces/energies are assembled with damping (`C·u`) and external loads (`W`). A Newton loop on reduced DOFs (excluding Dirichlet nodes) updates positions until residual < tol, ensuring equilibrium and stability for stiff dynamics.

Pseudocode

```
function
objfun(q_guess,u_prev,dt,tol,max_iter,M,EI,EA,W,C,ΔL,free):
  q ← q_guess
  repeat k=1..max_iter:
     compute edge vectors d_e, lengths L_e, tangents t_e = d_e /
L_e
    for each interior vertex v with edges e,f:
      dot = clamp(t_e·t_f, -1+ε, 1-ε)
      kb  = 2 * cross(t_e,t_f) / (1 + dot)
      chi = sqrt(2 * (1 + dot))
      add_bending(EI, kb, chi, ... → F_int, K)
    for each edge e:
      strain = (L_e - ΔL)/ΔL
      add_axial(EA, strain, ... → F_int, K)
    F_damp = -C*u_prev
    R = M*((q - q_guess)/dt) - (W + F_damp - F_int)
```

```
    reduce R,K to free DOFs → Rr, Kr
    if ||Rr|| < tol: return q, 0
    Δq_free = solve(Kr, -Rr);  q[free] += Δq_free
  return q, -1
```

(ii) Time Stepping

Each step forms the middle-node target `(x*,y*)`, computes a PID command for the end-effector, applies Dirichlet constraints, calls `objfun()` to get the next configuration, then updates velocities via backward difference. This implicit scheme (backward Euler) is robust for the rod's stiffness and for sudden load/control changes.

Pseudocode

```
q = q0; u = u0; t = 0
for step = 1..Nsteps:
  (x*,y*) = target(t)
   (x_c,y_c,θ_c,  I)  =  PID(x*,y*,  q[mid],  q_prev[mid],  I,  dt,
Kp,Ki,Kd)
  (x_c,y_c) = rate_limit((x_c,y_c), q[tip], max_step)
  apply_Dirichlet(q,  x_c,y_c,  θ_c,  ΔL)     # sets tip and tip-1
nodes
  fixed = left_clamp ∪ right_end;  free = all \ fixed
    (q_new,  err)  =  objfun(q,u,dt,tol,max_iter,M,EI,EA,W,C,ΔL,
free)
  if err<0: break
  u = (q_new - q)/dt;  q = q_new;  t += dt
```

(iii) Enforcement of Dirichlet Constraints

The right-end node is set to `(x_c,y_c)` and its neighbor to `(x_c-ΔL cosθ_c, y_c-ΔL sinθ_c)` before solving; the left end is clamped. These four right-end DOFs plus the left clamp are excluded from the solve, guaranteeing full rank and geometric consistency during Newton iterations.

Pseudocode

```
function apply_Dirichlet(q, x_c,y_c, θ_c, ΔL):
  q[Nx]   = x_c
  q[Ny]   = y_c
  q[N-1x] = x_c - ΔL * cos(θ_c)
  q[N-1y] = y_c - ΔL * sin(θ_c)

fixed_right = {Nx,Ny,N-1x,N-1y}
fixed = fixed_left ∪ fixed_right
free  = all_DOFs \ fixed
```

(iv) Path-Planning / Control Law (tracking error → {x_c, y_c, θ_c})

A PID maps middle-node error `e = (x*-x_m, y*-y_m)` to incremental end-effector commands. The derivative uses middle-node finite differences; integral removes steady bias. A per-step rate limiter caps `{x_c,y_c}` motion (feasibility + solver stability). In the submitted code, `θ_c` is held at 0, but the same pattern can drive it if desired.

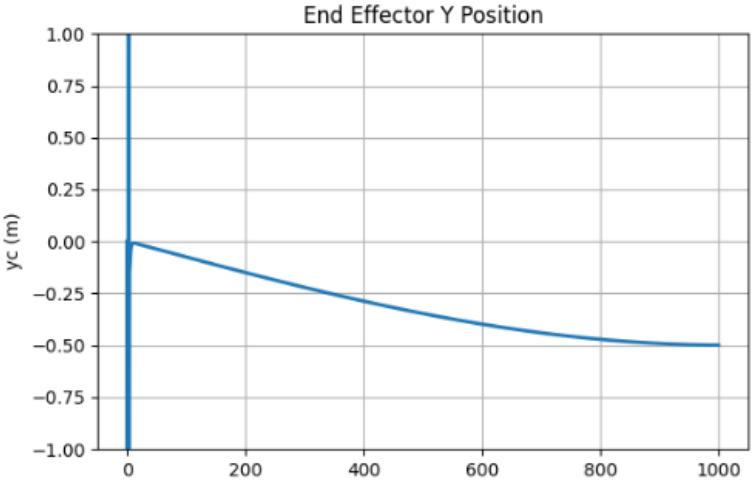
Pseudocode

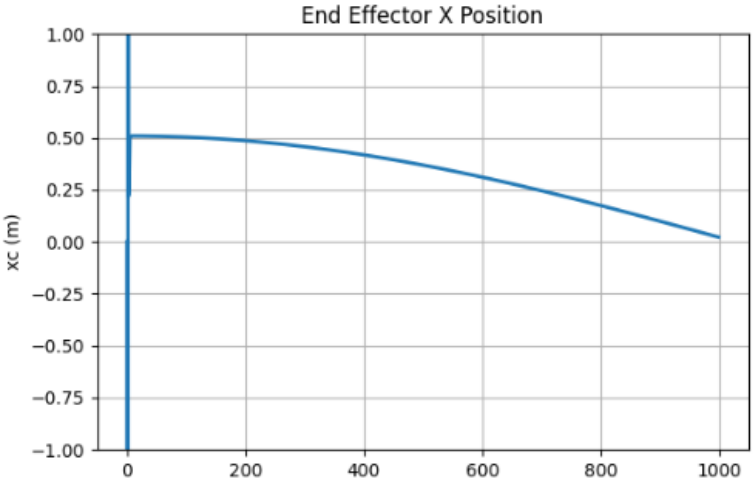
```
state: I = (Ix,Iy), prev mid = (x_m^-, y_m^-), prev cmd = (x_c^-, y_c^-)

function PID(x*,y*, x_m,y_m, x_m^-,y_m^-, I, dt, Kp,Ki,Kd):
  e  = (x* - x_m, y* - y_m)
```

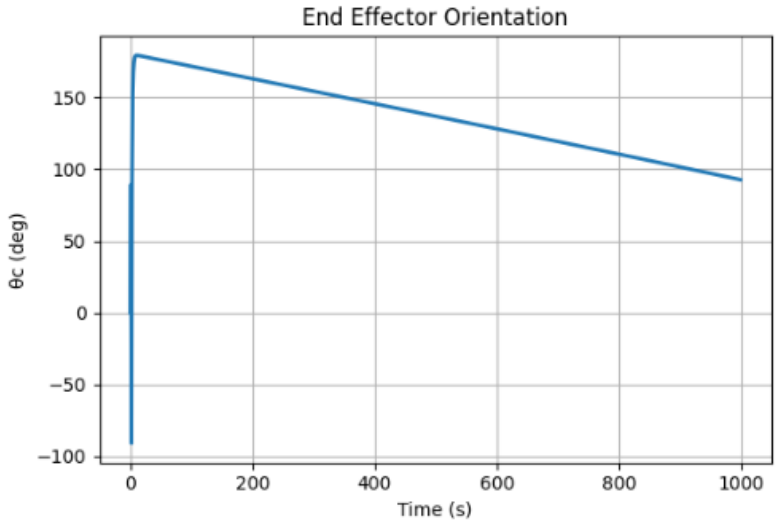
```
   ed = ((x_m - x_m^-)/dt, (y_m - y_m^-)/dt)   # measurement
derivative
  I  = (Ix + e.x*dt, Iy + e.y*dt)
  Δc = (Kp*e.x + Ki*Ix - Kd*ed.x,
        Kp*e.y + Ki*Iy - Kd*ed.y)
  x_c = x_c^- + Δc.x
  y_c = y_c^- + Δc.y
  θ_c = 0.0
  return (x_c,y_c,θ_c, I)


function rate_limit((x_c,y_c), (x_tip,y_tip), max_step):
  d = (x_c - x_tip, y_c - y_tip);  m = norm(d)
   if m > max_step: d = d*(max_step/m);  (x_c,y_c) = (x_tip +
d.x, y_tip + d.y)
  return (x_c,y_c)
```

---

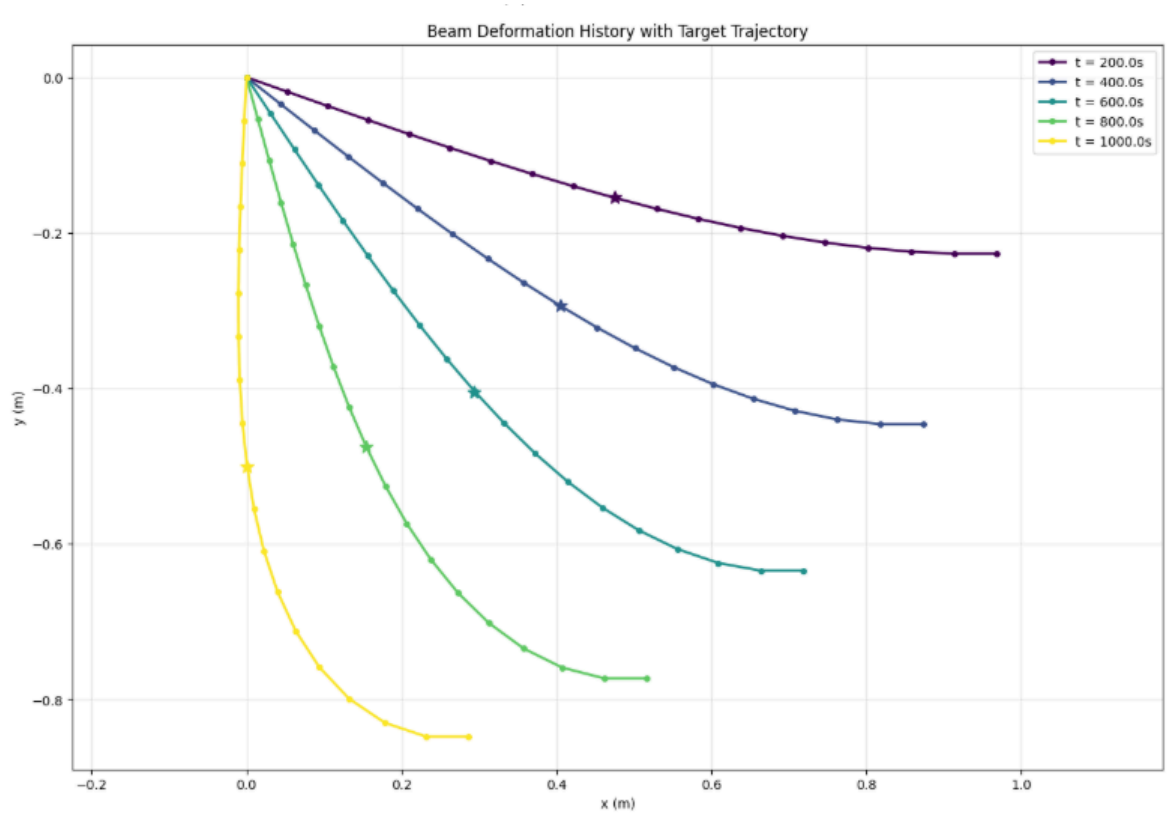
2. Plots of the control inputs xc(t), yc(t), θc(t) over time.

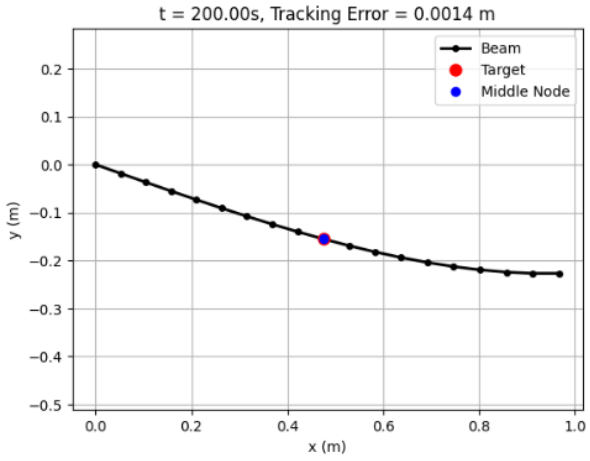


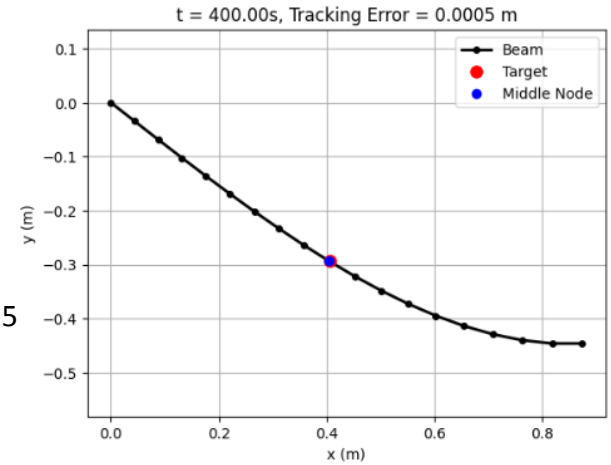


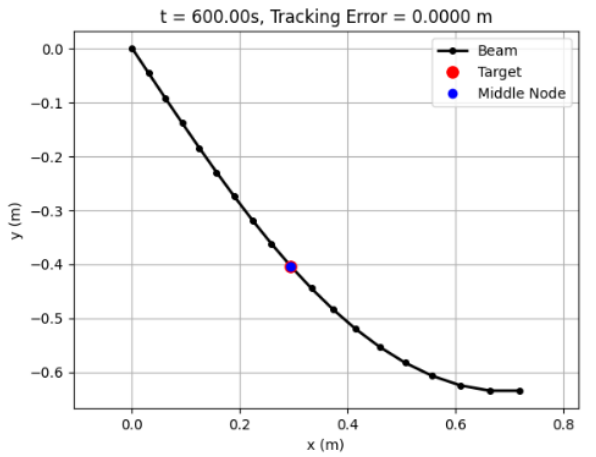


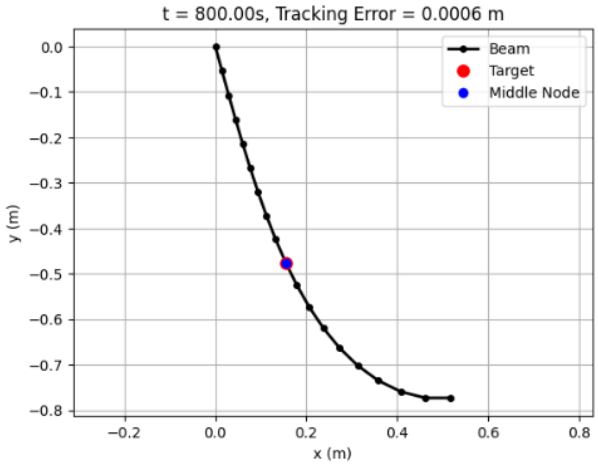
3. At least five snapshots of the beam shape (node positions) during the motion.

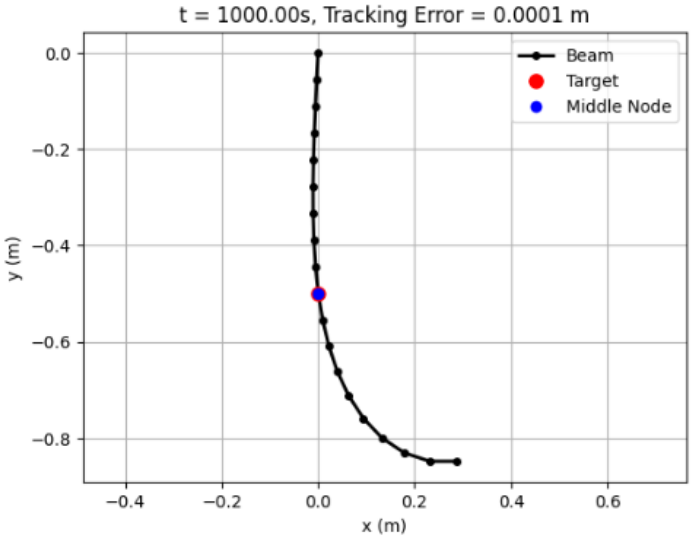












4. A brief discussion of feasibility limits due to the robot's workspace and joint limits; explain how you handle infeasible commands (e.g., saturation or trajectory re-timing).

The end-effector commands are constrained by the robot's physical workspace, joint angles, and actuator speed limits. After the PID controller computes `{x_c, y_c, θ_c}`, the code first applies a rate limiter that caps the motion per step (e.g., ≤ 2 mm), effectively re-timing the trajectory to maintain feasibility and ensure the beam solver converges. Next, the commands are saturated within predefined workspace and orientation bounds (e.g., −2 m ≤ x_c,y_c ≤ 2 m, |θ_c| ≤ θ_max), and when saturation occurs, an anti-windup correction adjusts the integral term to prevent drift. If convergence still fails, the system adaptively reduces `dt` or PID gains, preventing unachievable jumps. These measures keep all control inputs physically realizable while maintaining solver stability.